

What This Platform Does

It is a **SaaS web app for stock fundamental analysis**.

Users upload financial statements (PDF/Excel/CSV) or search for a company, and the system uses **AI to generate professional equity research reports**.

It also allows users to **ask follow-up questions in a chat interface** and (for premium users) **screen stocks using financial filters**.



How the System Is Built (Tech Stack)

Think of it as a 3-layer brain:



Frontend (User Interface)

Built using:

- React + TypeScript
- TailwindCSS
- Zustand/Redux for state management

This is the dashboard where users upload files, read reports, and chat with the AI.



Backend (Logic & AI Processing)

Built using:

- FastAPI (Python)
- PostgreSQL database
- Redis (caching)
- Celery (background tasks)
- LangChain + OpenAI/Claude (AI analysis)

This layer:

- Parses uploaded documents
- Sends data to the LLM for analysis
- Stores reports
- Handles chat Q&A using RAG (retrieval from documents)



AI & Data Intelligence

- Financial documents are split into chunks
- Converted into vector embeddings
- Stored in a vector database (Chroma/Pinecone)
- Used to answer user questions accurately from source documents



Main Features



AI Financial Report Generation

Users upload financial statements → system generates:

- Profitability analysis
- Liquidity & solvency checks
- Ratio calculations (ROE, margins, debt, etc.)
- Red flags & strengths
- Investment-style summary (with disclaimers)



Interactive AI Chat (Q&A)

Users can ask:

“Why did revenue decline?”

AI answers using the actual financial documents and cites sources.



Company Search

Users can search stocks and see financial metrics like:

- P/E ratio
- Revenue growth
- Profit margin
- Market cap



Stock Screener (Premium Feature)

Premium users can filter stocks using conditions like:

- $P/E < 30$
- Revenue growth $> 15\%$
- Sector = Technology
- Market Cap $> \$10B$

The system returns matching stocks ranked by relevance.



Database Structure (What Gets Stored)

Key tables include:

- **users** → account info & subscription tier
- **reports** → AI-generated analysis
- **source_documents** → uploaded files
- **stocks** → financial metrics for screening
- **screeners** → saved filter templates
- **chat_sessions** → conversation history



How Data Flows

1. User uploads a file
2. File stored in cloud storage
3. Text extracted & cleaned
4. Split into chunks
5. Embeddings generated
6. AI creates structured analysis
7. Report stored & shown to user
8. Chat uses stored document data to answer questions

Phase 1 — Build the Core (MVP)

This is the **minimum working product**.



Step 1: Create the Web App Interface

We will build a frontend where users can:

- Sign up / Log in
- Upload a financial PDF or Excel file
- See a generated report

Tech we use:

- React
- Tailwind CSS

At this stage, the UI is just a **smart shell waiting for a brain**.



Step 2: Build the Backend Server

We create a backend using **FastAPI (Python)** that can:

- Accept uploaded files
- Store them
- Send text to the AI
- Save the AI report

This becomes the **central nervous system** of the platform.



Step 3: Read Financial Files

We teach the system to:

- Extract text from PDFs
- Read tables from Excel/CSV
- Clean the financial data

Now the app can **understand raw financial statements**.



Step 4: Add the AI Analyst

Here we plug in GPT to act like a **professional equity analyst**.

The AI will generate:

- Profitability analysis
- Liquidity & debt analysis
- Financial ratios
- Red flags
- Strengths
- Final summary

Now the platform can **think and write like an analyst**.



Step 5: Save Reports in Database

We store:

- User info
- Uploaded files
- AI reports

So users can come back later and see past analysis.

Now the system has **memory**.



Step 6: Add AI Chat (Q&A on Report)

Users can ask:

“Why is debt high?”

“Is revenue growing?”

The system:

- Searches inside the uploaded document
- Uses AI to answer from the actual data

Now the platform becomes **interactive, not just static**.



Phase 2 — Add Smart Investing Tools

After the core works, we upgrade.



Step 7: Build Stock Database

We collect stock metrics like:

- P/E ratio
- Revenue growth
- Profit margin
- Market cap

Now the system knows **market data**, not just uploaded files.



Step 8: Build Stock Screener (Premium Feature)

Users can filter stocks like:

- $P/E < 25$
- $Growth > 10\%$
- Sector = Tech

The system returns matching companies.

Now users can **discover stocks**, not only analyze them.

What the Frontend Does

It provides screens for:

◆ Dashboard

Where users:

- Search a company
- Upload financial statements
- See previous reports

◆ Report Page

Shows:

- AI-generated financial analysis
- Key ratios (ROE, debt, margins)
- Red flags & strengths
- A chat box for questions

◆ Stock Screener (Premium)

Users set filters like:

- P/E ratio
- Revenue growth
- Sector

Frontend does **no heavy thinking**. It just sends user actions to the backend and displays results.



4. Backend Layer (System Brainstem)

Built using **FastAPI (Python)**.

The backend:

1. Receives file uploads
2. Stores files
3. Extracts financial data
4. Sends data to AI
5. Saves reports
6. Handles chat questions
7. Fetches stock data

It is like the **central manager**.

5. AI Layer (The Analyst Brain)

This is the most important part.

The system uses **LLMs like GPT-4 or Claude** through LangChain.

There are **two AI jobs**:



A. Report Generation AI

When a file is uploaded, the AI is instructed to act like:

“A professional equity research analyst with 15+ years experience.”

It analyzes:

Financial Health

- Profitability (margins, ROE)

- Liquidity (current ratio)
- Solvency (debt levels)

Ratio Analysis

- ROE, ROA, margins
- Debt-to-equity
- Efficiency ratios

Performance Drivers

Why revenue increased or declined

Red Flags

High debt, falling margins, weak cash flow

Strengths

Strong brand, high growth, competitive advantage

Investment Assessment

Risk vs reward (without giving financial advice)

So AI transforms **raw numbers** → **structured financial insight**.

B. Chat Q&A AI (RAG System)

When a user asks:

“Why did profits drop?”

The system:

1. Searches inside uploaded financial documents
2. Finds relevant text sections
3. Sends them to AI as context
4. AI answers based only on that data

This is called **Retrieval Augmented Generation (RAG)**.

It prevents AI from guessing and keeps answers grounded in real data.

7. File Processing Pipeline (Very Important)

When a PDF is uploaded:

- 1 File stored in cloud
- 2 Text extracted
- 3 Tables detected
- 4 Text cleaned
- 5 Split into chunks
- 6 Converted into embeddings (vectors)
- 7 Stored in vector database
- 8 Sent to AI for report generation

This pipeline converts **documents** → **AI-readable knowledge**.

8. Stock Screener Engine (Premium Feature)

The system maintains a **stock database** with metrics like:

- P/E ratio
- Revenue growth
- Profit margin
- ROE
- Market cap `Fundamental Analysis Platform -...`

Users apply filters:

- $P/E < 30$
- $Growth > 10\%$
- $Sector = Tech$

Backend runs a query and returns matching stocks ranked by relevance.

This turns the platform from **analysis tool** → **discovery tool**.



What is RAG?

RAG = Retrieval Augmented Generation

It is a method that makes AI answers come from **real documents**, not just memory.

Without RAG

AI answers from what it was trained on → may guess, may hallucinate

With RAG

AI first **searches your documents**, then answers using only that information

So RAG turns AI from a **dreamer** into a **librarian with receipts**.



The Core Idea

RAG has **two brains working together**:

Part	Job
Retrieval	Find relevant information from documents
Generation	Use that information to write the answer

First we **find facts**, then we **form the answer**.



Full RAG Flow (Step by Step)

Let's say a user uploads a company annual report and asks:

“Why did revenue decline in Q3?”

Here's what happens behind the scenes:



Step 1: Documents Are Prepared (Before Any Question)

When the PDF is uploaded, the system:

1. Extracts text from the PDF
2. Cleans it (removes headers/footers)
3. Splits it into small chunks (like paragraphs)

4. Converts each chunk into a **vector embedding**
5. Stores these vectors in a **vector database**

Now the system has a **searchable brain made of document meaning**, not just words.



What is an Embedding?

An embedding is a list of numbers that represents the **meaning** of text.

Example idea:

Text: “Revenue fell due to supply chain issues”

Embedding: [0.12, -0.98, 0.44, ...]

Similar sentences get **similar number patterns**.

So the system can search by *meaning*, not exact keywords.

? Step 2: User Asks a Question

User types:

“Why did revenue decline?”

This question is also converted into an embedding.

Now we have:

- Embedding of question
- Embeddings of document chunks



Step 3: Retrieval (Semantic Search)

The system compares the question vector with all document vectors.

It finds the **most similar chunks**, such as:

- “Revenue declined due to semiconductor shortages”
- “Q3 sales were affected by logistics disruptions”

This is called **semantic similarity search**.

Instead of searching for the word “decline”, it searches for **related meaning**.

Top 3–5 relevant chunks are selected.



Step 4: Context Building

Now the system builds a prompt for the AI like this:

Context from documents:

“Q3 revenue fell 8% due to supply chain disruptions in Asia...”

“Component shortages delayed product shipments...”

User Question:

“Why did revenue decline in Q3?”

So now AI is not blind. It has **evidence in front of it**.



Step 5: Generation (AI Writes Answer)

The LLM now answers using only the retrieved context.

Example answer:

“Revenue declined primarily due to supply chain disruptions and semiconductor shortages, which delayed shipments and reduced sales volume in Q3.”

This answer is **grounded in real document text**.